

Practica 9 - Excepciones

Objetivo: Conocer e interpretar los distintos modelos de excepciones que implementan los lenguajes de programación.

Ejercicio 1: ¿Explique claramente a qué se denomina excepción?

Una excepcion es una condicion inesperada o inusual que ocurre durante la ejecucion de un programa y no puede ser manejada en el contexto local.

Las excepciones denotan un comportamiento indeseable, pero es necesario controlarlo.

Una excepcion rompe el flujo normal de ejecucion y ejecuta un controlador de excepciones registrado previamente.

Ejercicio 2: ¿Qué debería proveer un lenguaje para el manejo de las excepciones? ¿Todos los lenguajes lo proveen?

Para que un lenguaje trate excepciones, debe proveer minimamente:

- Un modo de definirlas
- Una forma de reconocerlas
- Una forma de lanzarlas y capturarlas

No todos los lenguajes tienen manejo de excepciones, por ejemplo el lenguaje C, no cuenta con manejo de excepciones. En C, el control de errores se hace mediante valores de retorno, codigos de erros y funciones como setjmp

Ejercicio 3: ¿Qué ocurre cuando un lenguaje no provee manejo de excepciones? ¿Se podría simular? Explique cómo lo haría

Cuando un lenguaje no provee manejo de excepciones estructurado (es decir no tiene bloques try-catch, ni palabras clave como throw o raise), el control de errores debe hacerse de forma manual.

Un ejemplo en C, puede ser el siguiente, el cual 'simulamos' un error al intentar dividir por cero, se retorna un valor -1

```
#include <stdio.h>

int dividir(int a, int b, int* resultado) {
    if (b == 0) return -1; // error
    *resultado = a / b;
    return 0;              // éxito
}
```

```

}

int main() {
    int r;
    if (dividir(10, 0, &r) != 0) {
        printf("Error: división por cero.\n");
    } else {
        printf("Resultado: %d\n", r);
    }
    return 0;
}

```

Ejercicio 4: Cuando se termina de manejar la excepción, la acción que se toma luego es importante.

1. ¿Qué modelos diferentes existen en este aspecto?

Los modelos de manejo de excepciones son:

Reasuncion	Terminacion
Maneja la excepcion y continua la ejecucion	Termina la ejecucion y maneja la excepcion
Se refiere a la posibilidad de retornar la ejecucion normal del programa despues de manejar una excepcion . El controlador de excepciones realiza las acciones necesarias para manejar la excepcion, y luego el programa continua su ejecucion a partir del punto donde se produjo la excepcion.	El controlador de excepciones realiza las acciones necesarias para manejar la excepcion, pero no se retorna al punto donde se produjo la excepcion , sino que continua su ejecucion a partir de la finalizacion del manejador.
Lenguajes que lo utilizan: PL / 1	Lenguajes que lo utilizan: ADA, C++, Java, Python, PHP

- Resumiendo:

- Modelo de terminacion: NO pueden continuar en el punto donde se produjo la excepcion, se transfiere el control a un manejador y NO continua en el punto donde se produjo la excepcion.
- Modelo de Reasuncion: El programa puede continuar la ejecucion en el mismo punto donde se lanzo la excepcion, despues de ser manejada.

2. Dé ejemplos de lenguajes que utilizan cada uno de los modelos presentados anteriormente. Por cada uno responda respecto de la forma en que trabaja las excepciones

1. ¿Cómo se define?
2. ¿Cómo se lanza?
3. ¿Cómo se maneja?
4. ¿Cuál es su criterio de continuación?

◆ Modelo de Terminación (más común)

◆ Lenguaje: Java

a. ¿Cómo se define?

Las excepciones se definen como clases que heredan de `Throwable`, como `Exception` o `RuntimeException`.

b. ¿Cómo se lanza?

Con la palabra clave `throw`:

```
throw new IllegalArgumentException("Valor inválido");
```

c. ¿Cómo se maneja?

Usando bloques `try-catch`:

```
try {  
    metodoRiesgoso();  
} catch (IllegalArgumentException e) {  
    System.out.println("Error: " + e.getMessage());  
}
```

```
// otro codigo mas abajo....
```

d. ¿Cuál es su criterio de continuación?

No se regresa al punto del error. El flujo continúa después del bloque `try-catch`.

◆ Lenguaje: Python

- a. ¿Cómo se define?

Como clases que heredan de `BaseException` o `Exception`.

- b. ¿Cómo se lanza?

Con `raise`:

```
raise ValueError("Dato incorrecto")
```

c. ¿Cómo se maneja?

Con try-except:

```
try:
    funcion_riesgosa()
except ValueError as e:
    print("Error:", e)
```

d. ¿Cuál es su criterio de continuación?

También es de **terminación**. No se vuelve al punto donde se lanzó la excepción.

◆ Modelo de Reasuncion (menos común)

◆ Lenguaje: PL / 1

a. ¿Cómo se define una excepción en PL/I?

PL/I no usa clases como Java, sino que define condiciones como `ON ERROR`, `ON ZERODIVIDE`, `ON ENDFILE`, etc., que capturan eventos de error.

En PL / 1, las excepciones se definen como:

```
ON <Condicion> BEGIN;
    .....
END;
```

b. ¿Cómo se lanza una excepción?

Se lanza de forma **implícita** cuando ocurre una condición anormal durante la ejecución, por ejemplo una división por cero, lectura de fin de archivo, etc.

c. ¿Cómo se maneja?

Se usa la instrucción `ON` para declarar un manejador **asociado a una condición**.

Cuando ocurre la excepción, **se ejecuta ese bloque de código**, y luego se **retorna al punto donde ocurrió el error**, ya corregido.

d. ¿Cuál es su criterio de continuación?

→ **Reasunción (resumption)**: luego de manejar el error, se **retoma la ejecución exactamente en el punto donde falló**, pero con el contexto corregido por el manejador.

3. ¿Cuál de esos modelos es más inseguro y por qué?

El modelo de reasunción es el mas inseguro porque:

- Puede ocultar errores graves al permitir continuar la ejecución.
- Es difícil asegurar que el estado del programa esté consistente después de la excepción.
- Complica el mantenimiento y comprensión del flujo de ejecución.

Por eso, los lenguajes de programación modernos como Java o Python, prefieren usar el modelo de terminación, ya que se maneja la excepción pero no se vuelve a ejecutar el punto de error.

Ejercicio 5: La propagación de los errores, cuando no se encuentra ningún manejador asociado, no se implementa igual en todos los lenguajes. Realice la comparación entre el modelo de Java, Python y PL/I, respecto a este tema. Defina la forma en que se implementa en un lenguaje conocido por Ud.

Lenguaje	¿Propaga si no hay manejador?	¿Cómo termina si no se maneja?	¿Modelo?
Java	Sí. Usa <code>throw</code> y <code>try-catch</code> .	Imprime stack trace y termina.	Terminación
Python	Sí. Usa <code>raise</code> y <code>try-except</code> .	Muestra traceback y termina.	Terminación
PL/I	No necesariamente. Puede manejarse con <code>ON</code> .	Si no hay <code>ON</code> , puede abortar o depender del sistema.	Reasunción (continuación)

Pongo un ejemplo de división por cero en Java.

```
public static void main(String[] args) {  
    int numerador = 10;  
    int denominador = 0;  
  
    try {  
        int resultado = numerador / denominador;  
        System.out.println("Resultado: " + resultado);  
    } catch (ArithmeticException e) {  
        System.out.println(";Error! División por cero.");  
        // Corrección: asignar un nuevo valor al denominador  
        denominador = 1;  
        int resultadoCorregido = numerador / denominador;  
        System.out.println("Resultado corregido: " + resultadoCorregido);  
    }  
}
```

```
}  
}
```

También se puede agregar un bloque Finally, el cual es código que se va a ejecutar SIEMPRE.

Ejercicio 6: Sea el siguiente programa escrito en Pascal

```
...  
Procedure Manejador;  
  Begin ... end;  
Procedure P(X:Proc);  
  begin  
    ....  
    if Error then X;  
    ....  
  end;  
Procedure A;  
  begin  
    ....  
    P(Manejador);  
    ....  
  end;  
....
```

¿Qué modelo de manejo de excepciones está simulando? ¿Qué necesitaría el programa para que encuadre con los lenguajes que no utilizan este modelo? Justifique la respuesta.

El programa en pascal, está simulando un modelo de Reasuncion, porque:

- La excepcion no termina el flujo de ejecucion.
- En lugar de abortar o interrumpir el programa, se llama al procedimiento manejador x y luego continua la ejecucion.
- El punto de la ejecucion no se pierde, se retorna despues del llamado al manejador. Entonces el comportamiento sumula el modelo de continuacion por reasuncion, como el que usa PL/1, donde el manejador puede corregir el estado y permitir continuar en el mismo punto del programa.

Para que el programa encuadre con los lenguajes que no utilizan este modelo (como Java o Python) debería:

- No permitir continuar la ejecucion en el mismo lugar donde ocurrio el error (FACTOR IMPORTANTISIMO que tienen los lenguajes con excepciones de modelo con terminacion.)
- Interrumpir el flujo actual y pasar el control a un bloque manejador superior, como un catch (en java) o except (en Python)
- Evitar la reasuncion automatica.

a) Una transformacion conceptual en Pascal puede ser la siguiente:

```
Procedure P;  
Begin  
    ...  
    If Error Then  
        RaiseException; { O lanzar error que finaliza el flujo actual }  
    ...  
End;  
  
Procedure A;  
Begin  
    Try  
        P;  
    Catch  
        Manejador;  
    End;  
End;
```

b)

```
public class Principal {  
  
    static int x; // variable global  
  
    public static void main(String[] args) {  
        x = 4;  
        boolean b1 = false;  
        boolean b2 = true;  
  
        if (!b1) {  
            manejador2(); // modifica x global  
        }  
  
        try {  
            p(b1); // puede lanzar excepci3n que simula Manejador1  
        } catch (Manejador1Exception e) {  
            // se maneja la excepci3n lanzada desde P()  
            System.out.println("Manejador1 se ejecut3.");  
        }  
  
        System.out.println("Valor final de x: " + x);  
    }  
  
    public static void p(boolean b1Param) throws Manejador1Exception {
```

```

    int x = 1; // variable local de P

    if (b1Param) {
        throw new Manejador1Exception(); // simula llamada a Manejador1
    }

    x = x + 4;
    System.out.println("x interno de P: " + x);
}

public static void manejador2() {
    x = x * 100;
}

// Definición de la excepción personalizada
static class Manejador1Exception extends Exception {
    public Manejador1Exception() {
        Principal.x += 1; // modifica la x global como en Pascal
    }
}
}

```

Ejercicio 8: Sean los siguientes procedimientos de un programa escrito en Java.

```

public static void main (String[] argos){
    Double array_doubles[]= new double[500];
    for (int i=0; i<500; i++){
        array_doubles[i]=7*i;
    }
    for (int i=0 ; i<600 ; i=i+25){
        try{
            system.out.println("El elemento en "+ i + " es " + acceso_por_indice (array_doubles,i));
        }
        catch(ArrayIndexOutOfBoundsException e){
            system.out.println(e.toString());
        }
        catch(Exception a){
            system.out.println(a.toString());
        }
        finally{
            system.out.println("sentencia finally");
        }
    }
}

Public static double acceso_por_indice (double [] v, int indice) throws Exception; ArrayIndexOutOfBoundsException{
    if ((indice>=0) && (indice<v.length)){
        Return v[indice];
    }
    else{
        if (indice<0){
            // caso excepcional
            Throw new ArrayIndexOutOfBoundsException(" el índice" + indice + " es un número negativo");
        }
        else{
            // caso excepcional
            Throw new Exception(" el indice" + indice + " no es una posición válida");
        }
    }
}

```

a) Analizar el ejemplo y decir qué manejadores ejecuta y en qué valores quedan las variables. JUSTIFIQUE LA RESPUESTA.

Se declara: array_doubles de 500 elementos.

En el primer for: la variable array_doubles se inicializa con el valor $7 * i$ para cada posición de i, que iría de 0 a 499

En el segundo for: Se llama al método acceso_por_indice(array_doubles, i) con i tomando valores: 0, 25, 50, ..., 475, 500, 525, 550, 575 .

dentro del for, mientras esta en el rango de 0..475, los índices son válidos.

Cuando $i = 500$ o mayor, se lanza una Excepción Exception , ya que la variable i , está fuera del rango de índices del vector.

Lo que podemos decir, es que solamente se lanza la Excepción Exception , se lanza cuando $i=500$, $i=525$, $i =550$, $i = 575$,

Estas excepciones son captadas por el manejador catch (Exception e)

Mientras que el **manejador `ArrayIndexOutOfBoundsException`** nunca se ejecuta, ya que **en ningun momento i toma valores negativos.**

Otra cosa que tenemos para decir, es que el bloque `finally`, se ejecuta cada iteracion del `for`, ocurra o no una excepcion, lo que este dentro de un bloque `finally` SIEMPRE se va a ejecutar.

Valores finales:

- La variable `array_doubles` : Queda todo inicializado, con valores de $7 * i$, desde $i=0$ hasta $i=499$
- La variable `i` termina el `for` con valor 600.

b) La excepción se propaga o se maneja en el mismo método? ¿Qué instrucción se agrega para poder propagarla y que lleve información?.

La excepcion se lanza en el metodo `acceso_por_indice`, pero se propaga hacia el `main`, el bloque `catch(Exception e)` es el encargado de captar esa excepcion.

Con respecto a la instruccion que se agrega para propagarla y que lleve informacion, es la instruccion:

```
Throw new Exception(" el indice" + indice + " no es una posición válida");
```

Ya que el `throw` lo que hace es lanzar una excepcion, y como en el metodo que se lanza la excepcion no hay ningun `try-catch`, se propaga hacia el `main`.

c) como modificaría el método “acceso_por_indice” para que maneje él mismo la excepción.

Para que el método `acceso_por_indice` maneje él la excepción, pondría el `try-catch` que está en el `main` dentro de ese método, y dentro del `try` tendría todo el código original que se encuentra, y ademas sacaria el `throws` que aparece en la firma del metodo.

Ejercicio 9: Indique diferencias y similitudes entre Phyton y Java con respecto al manejo de excepciones.

Similitudes:

1. Modelo de manejo:

Ambos lenguajes utilizan el **modelo de Terminación**, lo que significa que cuando ocurre una excepción, el flujo normal del programa se interrumpe y se transfiere al manejador de excepciones correspondiente. Luego, **no se retoma el punto donde ocurrió la excepción.**

2. Bloque try :

Tanto en Java como en Python, las excepciones se manejan utilizando un bloque `try` que contiene el código que puede generar errores.

3. Bloque finally :

Ambos lenguajes permiten el uso de un bloque `finally`, el cual **se ejecuta siempre**, ocurra o no una excepción, y sirve generalmente para liberar recursos o cerrar conexiones.

! Diferencias:

Característica	Java	Python
Palabra clave para manejar	<code>catch</code>	<code>except</code>
Palabra clave para lanzar	<code>throw</code>	<code>raise</code>
Cláusula adicional	No tiene <code>else</code> en excepciones	Tiene <code>else</code> , se ejecuta si no hubo excepción en <code>try</code>
Obligatoriedad del manejo	En Java, algunas excepciones deben ser atrapadas o declaradas con <code>throws</code> (Checked Exceptions)	En Python todas son Unchecked , no es obligatorio atraparlas
Sintaxis del catch/except	<code>catch (TipoExcepcion e)</code>	<code>except TipoExcepcion as e:</code>

Comparacion en codigo:

```
try {
    int resultado = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: división por cero.");
} finally {
    System.out.println("Este bloque siempre se ejecuta.");
}
```

```
try:
    resultado = 10 / 0
except ZeroDivisionError as e:
    print("Error: división por cero.")
finally:
    print("Este bloque siempre se ejecuta.")
```

Ejercicio 10: ¿Qué modelo de excepciones implementa Ruby? ¿Qué instrucciones específicas provee el lenguaje para el manejo de excepciones y cómo se comportan cada una de ellas?

Modelo de excepciones:

Ruby implementa el **modelo de terminación** para manejo de excepciones, al igual que Java y Python. Es decir, cuando ocurre una excepción, **se abandona la ejecución del bloque actual y se transfiere el control a un manejador**.

Instrucciones específicas de Ruby:

1. `begin - rescue - else - ensure`:

- `begin` : bloque que contiene el código que puede generar excepciones.
- `rescue` : captura la excepción y permite manejarla.
- `else` : (opcional) se ejecuta si **no ocurrió ninguna excepción**.
- `ensure` : (similar a `finally` en Java/Python) se ejecuta **siempre**, ocurra o no una excepción.

```
begin
  # Código que puede generar una excepción
  resultado = 10 / 0
rescue ZeroDivisionError => e
  puts "Ocurrió una excepción: #{e.message}"
else
  puts "No ocurrió ninguna excepción."
ensure
  puts "Esto se ejecuta siempre."
end
```

Ejercicio 11: Indique el mecanismo de excepciones de JavaScript

Modelo de excepciones:

JavaScript también utiliza el **modelo de terminación**. Cuando ocurre una excepción, se interrumpe el flujo normal del programa y se transfiere el control al bloque de manejo.

Instrucciones de manejo de excepciones en JavaScript:

1. `try - catch - finally`:

- `try` : contiene el código que puede fallar.
- `catch` : captura y maneja cualquier error que ocurra en el bloque `try`.
- `finally` : se ejecuta siempre, sin importar si ocurrió un error o no.

```

try {
  let resultado = 10 / 0;
  throw new Error("Error personalizado");
} catch (error) {
  console.log("Ocurrió una excepción: " + error.message);
} finally {
  console.log("Este bloque siempre se ejecuta.");
}

```

Para lanzar excepciones, se utiliza la instrucción `throw`.

Se puede lanzar cualquier tipo de dato, aunque lo más común es lanzar objetos de tipo `Error` o sus derivados (`TypeError`, `RangeError`, etc.).

A diferencia de Java, en JavaScript **no hay excepciones "obligatorias"** (checked exceptions). Todas son **unchecked**.

Ejercicio 12: Sea el siguiente programa escrito en PYTHON::Indique el camino de ejecución.

```

def dividir():
    x = a / b
    print (("Resultado"), (x))

while True:
    try:
        a = int(input("Ingresa el primer numero: \n"))
        b = int(input("Ingresa el segundo numero: \n"))
        dividir()
        break
    except ZeroDivisionError:
        print ("No se permite dividir por cero")

finally:
    print ("Vuelve a probar")

```

a) Describa qué caminos ejecuta para diferentes valores de ingreso

- Si se ingresan caracteres numericos:
 - Si b es distinto de 0, el programa
 - Llama al metodo dividir, el cual realiza la operacion e imprime el resultado de a /

- b.
 - Se ejecuta el bloque finally
 - El break hace que el bucle while termine. No itera nuevamente.
- Cuando B sea igual a 0, el programa
 - Lanza una excepcion, la cual es captada por el manejador except ZeroDivisionError.
 - Se ejecuta el bloque finally
 - Como el break no se ejecuta, vuelve a iterar sobre el bucle while, por lo tanto vuelve a pedir los numeros
- Si se ingresan valores no numericos (ejemplo letras):
 - Se produce una excepcion ValueError al intentar convertir input con int
 - Como no hay un except para el ValueError o cualquier otro tipo de excepcion, el programa se interrumpe, corta la ejecucion del mismo, y en este caso el finally no se ejecutaria.

b) Agregar el uso de una excepción anónima

```
#!/usr/bin/env python
# calc.py
def dividir():
    x = a / b
    print("Resultado", x)

while True:
    try:
        a = int(input("Ingresa el primer numero: \n"))
        b = int(input("Ingresa el segundo numero: \n"))
        dividir()
        break
    except ZeroDivisionError:
        print("No se permite dividir por cero")
    except: # Excepcion anonima
        print("Error inesperado")
    finally:
        print("Vuelve a probar")
```

Ejercicio 13: Sea el siguiente código escrito en JAVA

```

public class ExcepcionUno extends Exception {

    public ExcepcionUno(){
        super(); // constructor por defecto de Exception
    }

    public ExcepcionUno( String cadena ){
        super( cadena ); // constructor param. de Exception
    }

}

public class ExcepcionDos extends Exception {

    public ExcepcionDos(){
        super(); // constructor por defecto de Exception
    }

    public ExcepcionDos( String cadena ){
        super( cadena ); // constructor param. de Exception
    }

}

public class ExcepcionTres extends Exception {

    public ExcepcionTres(){
        super(); // constructor por defecto de Exception
    }

    public ExcepcionTres( String cadena ){
        super( cadena ); // constructor param. de Exception
    }

}

public class Lanzadora {

    public void lanzaSiNegativo(int param) throws ExcepcionUno {
        if (param < 0)
            throw new ExcepcionUno("Numero negativo");
    }

    public void lanzaSiMayor100(int param) throws ExcepcionDos {
        if (param >100 and param<125)
            throw new ExcepcionDos("Numero mayor100");
    }

    public void lanzaSiMayor125(int param) throws ExcepcionTres {
        if (param >= 125)
            throw new ExcepcionTres("Numero mayor125");
    }

}

```

```

import java.io.FileInputStream;
import java.io.IOException;

public class Excepciones {

    public static void main(String[] args) {
        // Para leer un fichero

        Lanzadora lanza = new Lanzadora();
        FileInputStream entrada = null;
        int leo;
        try {
            entrada = new FileInputStream("fich.txt");
            while ((leo = entrada.read()) != -1){
                if (leo < 0)
                    lanza.lanzaSiNegativo(leo);
                else if (leo > 100)
                    lanza.lanzaSimayor100(leo);
            }

            entrada.close();
            System.out.println("Todo fue bien");
        }
        catch (ExcepcionUno e) { // Personalizada
            System.out.println("Excepcion: " + e.getMessage());
        }
        catch (ExcepcionDos e) { // Personalizada
            System.out.println("Excepcion: " + e.getMessage());
        }
        catch (IOException e) { // Estándar
            System.out.println("Excepcion: " + e.getMessage());
        }
        finally {
            if (entrada != null)
                try {
                    entrada.close(); // Siempre queda cerrado
                }
                catch (Exception e) {
                    System.out.println("Excepcion: " + e.getMessage());
                }
            System.out.println("Fichero cerrado.");
        }
    }
}

```

a) Indique cómo se ejecuta el código. Debe quedar en claro los caminos posibles de ejecución, cuales son los manejadores que se ejecutan y cómo se buscan los mismos y si en algún caso se produce algún error.

Caminos posibles de ejecución:

1: Archivo leído correctamente y todos los bytes ingresados están entre 0 y 100:

- No se lanza ninguna excepción
- Se cierra el archivo

- Se imprime "Todo fue bien" y "Fichero cerrado."

2: Si se lee un byte <0

- Se lanza ExcepcionUno
- Se atrapa en el manejador catch (ExcepcionUno e)
- Se imprime el mensaje "Excepcion: Numero negativo"
- Se ejecuta el finally y se cierra el archivo.

🟡 **Nota :** `entrada.read()` devuelve un `int` , pero nunca retorna valores < 0 excepto -1 para EOF. Así que este camino **no ocurrirá** realmente.

3: Si se lee un byte > 100 y < 125

- Se lanza ExcepcionDos
- Se atrapa en el manejador catch (ExcepcionDos e)
- Se imprime el mensaje "Excepcion: Numero mayor100"
- Se ejecuta el bloque finally y se cierra el archivo.

4: Si se lee un byte >= 125

- Se ejecuta el metodo `lanzaSiMayor100`
NO LANZA EXCEPCION, ya que no entra al if porque param seria >=125
- La lectura continua hasta EOF (es decir se sigue ejecutando el while)

5: Si el archivo `fich.txt` no existe o no puede abrirse:

- Se lanza una `IOException`
- Se atrapa en el manejador catch (`IOException e`)
- Se imprime mensaje de excepcion
- Se ejecuta finally.

El `while` solo sigue ejecutándose mientras los valores leídos estén entre 0 y 100 , inclusive. Si se lee un valor fuera de ese rango, se lanza una excepción y como Java utiliza **modelo de terminación**, el bucle se interrumpe inmediatamente y el flujo pasa al `catch` y luego al `finally` .

Ejercicio 14: Dado el siguiente código en Java. Indique todos los posibles caminos de resolución, de acuerdo a los números que vaya leyendo del archivo.

```

class ExcepcionE1 extends Exception {

    public ExcepcionE1(){
        super(); // constructor por defecto de Exception
    }

    public ExcepcionE1( String cadena ){
        super( cadena ); // constructor param. de Exception
    }

}

```

```

class ExcepcionE2 extends Exception {

    Public ExcepcionE2(){
        super(); // constructor por defecto de Exception
    }

    Public ExcepcionE2( String cadena ){
        super( cadena ); // constructor param. de Exception
    }

}

```

// Esta clase lanzará la excepción

```

public class Evaluacion {
    void Evalua( int edad ) throws ExcepcionE1, ExcepcionE2 {
        if ( edad < 18 )
            throw new ExcepcionE1( "Es una persona menor de edad" );
        else if ( edad > 70 )
            throw new ExcepcionE2( "Es persona mayor de edad" );
    }

    void Segmenta( int edad ) throws ExcepcionE1, ExcepcionE2 {
        if ( edad < 35 )
            throw new ExcepcionE1( "Es una persona joven" );
    }

}

```

```

class AnalisisEdadPoblacion{
    public static void main( String[] args ){
        // Para leer un fichero
        Evaluacion Invoca = new Evaluacion();
        FileInputStream ream entrada = null;
        int leo;
        try{
            entrada = new FileInputStream( "fich.txt" );
            while ( ( leo = entrada.read() ) != -1 ) {
                try {
                    if (leo<0) {
                        throw new ExcepcionE1( "Edad inválida" );
                    }
                    else{
                        if (leo>120){
                            throw new ExcepcionE1( "Edad inválida" );
                        }
                    }
                    invoca.evalua( leo);
                    invoca.segmenta( leo );
                    System.out.println( " ( "Es persona adulta, Todo fue bien" );
                }
                catch ( ExcepcionE2 e ){
                    System.out.println( "Excepcion: " + e.getMessage() );
                }
            }
        }
    }
}

```

```

    }
    catch ( ExcepcionE1 e ){
        System.out.println( "Excepcion: " + e.getMessage() );
    }
} catch (FileNotFoundException e1) {
    System.out.println("No se encontró el archivo");
} catch (IOException e) {
    System.out.println("Problema para leer los datos");
}finally {
    if (entrada != null)
        try {
            entrada.close();
        } catch (Exception e) {
            System.out.println("Excepcion: " + e.getMessage());
        }
    System.out.println("Fichero cerrado.");
}
}
}

```

Caminos de ejecucion:

1: Se intenta abrir el archivo `fich.txt`

- Si no existe, se lanza `FileNotFoundException`.
- Si hay problemas al leer, se lanza `IOException`.
- Cada excepcion tiene un manejador (`catch`) correspondiente

2: Se lee el archivo byte por byte con `entrada.read()`

- Cada valor leído se guarda en `leo`
- El bucle `while` sigue hasta que `leo == -1` (fin del archivo, o EOF)
- Por cada `leo`, se evalua el bloque interno `try`

3: Validaciones

- Si `leo < -` se lanza `ExcepcionE1` con mensaje "Edad Invalida"
- Si `leo >=120` se lanza `ExcepcionE1` con mensaje "Edad Invalida"
- en ambos casos, interrumpen el bucle interno y salta al `catch`.

4: Validaciones con metodos

Si no se lanzo ninguna excepcion, se llaman a los metodos:

- `invoca.evalua(leo)`:
 - Si `leo < 18` → lanza `ExcepcionE1` → "Es una persona menor de edad"
 - Si `leo > 70` → lanza `ExcepcionE2` → "Es persona mayor de edad"
- `invoca.segmenta(leo)`:
 - Si `leo < 35` → lanza `ExcepcionE1` → "Es una persona joven"

5: Caso todo fue bien

- Si leo esta entre 35 y 70 inclusive.
- No se lanza ninguna excepcion
- Se imprime: "Es persona adulta, todo fue bien."

6: Bloque Finally:

Se ejecuta SIEMPRE, por mas de que se haya lanzado o no una excepcion.

- Cierra el archivo (si se abrio)
- Imprime: "Fichero cerrado."

Conclusión

El `while` sigue ejecutándose en todos los casos, incluso si hay excepciones internas, porque están atrapadas con `try-catch`.

 El `while`, solo **se detiene cuando**:

1. `entrada.read()` devuelve `-1` (fin del archivo), o
2. Se lanza una excepción **no capturada** o **externa** (por ejemplo `IOException` afuera del bucle).